

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
IIC-3694 Inteligencia Artificial Generativa

Desafío de Investigación

Fecha Máxima de Entrega: Viernes 6 de Noviembre (11:59 pm)

Muchach@s, este semestre el desafío de investigación estará centrado en construir un entorno de entrenamiento (harness) y su dataset asociado, con el objetivo de dotar a un gran modelo de lenguaje (LLM) pequeño de mejores capacidades de razonamiento y/o uso de herramientas. En el desafío de curso el objetivo será la construcción del entorno y del dataset, mientras que luego, en el proyecto de curso, el objetivo será utilizarlos para entrenar el modelo y comparar distintas estrategias de entrenamiento.

Las capacidades más recientes de los modelos de frontera, como programación con agentes o creación de presentaciones, se han logrado en buena medida incorporando un harness a la lógica de entrenamiento. Es decir, se construye una base de código que estandariza cómo realizar una tarea, se ajusta el modelo para que aprenda a interactuar con ella y luego se entrena mediante aprendizaje reforzado (RL), ubicando un verificador al final de cada intento. El ciclo resultante para aprender una tarea es aproximadamente el siguiente:

1. Construir un harness.
2. Mediante prompting, hacer que un modelo interactúe con el harness.
3. Seleccionar las trazas correctas para hacer ajuste supervisado (SFT) del modelo, de modo que interiorice las reglas de interacción.
4. Evaluar el modelo ajustado frente al harness e identificar dónde sigue fallando.
5. Entrenarlo con RL (por ejemplo, RLVR/GRPO/PPO) frente al harness, usando el verificador como recompensa.
6. Iterar: revisar trazas, corregir el harness o el verificador (ex., ver si el modelo realiza reward hacking) y volver a entrenar.

Todo el ciclo depende de una pieza central: un buen verificador. Con él se generan los datos para todas las modalidades de entrenamiento: las trazas correctas son datos de SFT, los pares de respuestas correcta/incorrecta para un mismo problema son datos de DPO, y el verificador mismo es la recompensa para RLVR/GRPO/PPO. La idea central de este desafío es que el harness y los datos no son un paso aparte del entrenamiento, sino lo que lo hace posible.

I. Construcción del Entorno y del Dataset

El trabajo podrá ser realizado en grupos de hasta 4 personas. Cada grupo debe elegir un problema de razonamiento y/o uso de herramientas que un modelo pequeño (del orden de 1-2B parámetros, por ejemplo de la familia Qwen) resuelva mal, y construir:

1. Harness: formato de prompt, herramientas si aplica, parser de la respuesta y verificador. Todo debe seguir la interfaz común definida por el curso, del tipo

   reward(prompt, completion) -> float

   de modo que cualquier grupo pueda usar el entorno de otro. En el caso de uso de herramientas, se recomienda limitar la interacción a pocos turnos (por ejemplo, el modelo emite una llamada, el harness la ejecuta y le devuelve el resultado).

2. Tareas: con particiones de entrenamiento, validación y prueba (train/eval/test), niveles de dificultad y respuestas verificables por construcción, es decir, calculadas por código y no extraídas a mano.

3. Datos derivados: trazas para SFT generadas con un modelo abierto de mayor tamaño y filtradas con el verificador, y pares de preferencia para DPO/RLHF.

4. Línea base: medición de pass@1 y pass@k de varios modelos, incluido el modelo objetivo.

El punto 4 es clave para el entrenamiento con RLVR/GRPO/PPO: si el modelo pequeño nunca acierta, no hay señal y no aprende nada; si siempre acierta, no hay nada que aprender. Las tareas deben caer en la zona aprendible del modelo objetivo. Asimismo, dado que RLVR/GRPO/PPO es efectivo afinando capacidades que el modelo ya posee, pero no incorporando conocimiento nuevo, los problemas deben ser de razonamiento o uso de herramientas y no de memorización de datos.

Algunos ejemplos de problemas posibles son: aritmética o álgebra de varios pasos, acertijos lógicos con solución verificable, planificación o scheduling con restricciones, consultas SQL sobre una base de datos, manipulación de tablas, o uso de herramientas como una calculadora, un intérprete de Python restringido o un buscador sobre una colección local de documentos.

La calidad del entorno y del dataset será evaluada en función de 4 criterios:

i) Originalidad: novedad del problema elegido y del método utilizado para generar las tareas.

ii) Profundidad: volumen, diversidad y niveles de dificultad de las tareas.

iii) Robustez: que el verificador acepte respuestas equivalentes y no pueda ser engañado fácilmente.

iv) Calibración: evidencia de que las tareas son aprendibles para el modelo objetivo.

II. Evaluación del Entorno

Para la línea base deben utilizar modelos de frontera, privilegiando modelos abiertos. Dado que en esta etapa solo se necesita operar en modo inferencia y no entrenamiento, no se requerirá una gran capacidad de cómputo. Algunos modelos recomendados son los correspondientes a las familias: Qwen, Llama, OLMo, Mistral, DeepSeek, entre otros.

Un aspecto relevante será el diseño del verificador, el cual debe reconocer múltiples formas correctas de respuesta. Por ejemplo, ante la pregunta: ¿Cuántas sillas tiene la sala xyz?, respuestas como: “40”, “cuarenta”, “cuatro filas de 10 sillas”, o “20 adelante y 20 atrás” son correctas, pero son muy distintas en su sintaxis. A diferencia de una evaluación tradicional, el verificador será además la recompensa del entrenamiento, por lo que debe resistir intentos de engaño (reward hacking), como entregar varias respuestas a la vez, explotar el formato esperado o producir salidas que el parser interprete erróneamente como correctas. Se valorará incluir casos de prueba diseñados específicamente para intentar engañar al verificador.

III. Entregables

- Código del harness siguiendo la interfaz común del curso. Siguiendo una política de apertura, el entorno será compartido con el resto del curso, de modo que pueda ser utilizado por otros grupos en el proyecto.
- Descripción del entorno y del dataset, detallando el problema elegido, el método de generación de tareas, las particiones y niveles de dificultad, ejemplos representativos, y debilidades y fortalezas.
- Datos derivados: trazas para SFT y pares de preferencia para DPO, indicando el modelo y el procedimiento utilizados para generarlos.
- Informe de la línea base, indicando los modelos considerados, las métricas pass@1 y pass@k por nivel de dificultad, y ejemplos cualitativos que ilustren fortalezas y debilidades. Adicionalmente, un análisis de la robustez del verificador y de la calibración de las tareas para el modelo objetivo.

IV. Referencias

Para la construcción del harness y el dataset pueden ver: [3], [4],[2], [1], among others.

Bibliografía

[1] Abhishek Rao Abu Awal Md Shoeb Abubakar Abid Adam Fisch Adam R. Brown Adam Santoro Aditya Gupta Adrià Garriga-Alonso Agnieszka Kluska Aitor Lewkowycz Akshat Agarwal Alethea Power Alex Ray Alex Warstadt Alexander W. Kocurek Ali Safaya Ali Tazarv Alice Xiang et al. (430 additional authors not shown) Aarohi Srivastava, Abhinav Rastogi. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. In ICLR, 2023.

[2] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. In NEURIPS, 2021.

[3] Z. Stojanovski, O. Stanley, J. Sharratt, R. Jones, A. Adefioye, J. Kaddour, and A. Köpf. Reasoning gym: Reasoning environments for reinforcement learning with verifiable rewards. In NEURIPS, 2025.

[4] Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, Honglin Guo, Junzhe Wang, Xin Guo, Dingwen Yang, Chenyang Liao, Wei He, Songyang Gao, Lu Chen, Rui Zheng, Yicheng Zou, Tao Gui, Qi Zhang, Xipeng Qiu, Xuanjing Huang, Zuxuan Wu, and Yu-Gang Jiang. AgentGym: Evaluating and training large language model-based agents across diverse environments. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, NEURIPS, 2025.